

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
Национальный исследовательский ядерный университет «МИФИ»



**Институт
интеллектуальных кибернетических систем
Кафедра №22 «Кибернетика»**

Система автоматизированного развёртывания окружений

План разработки ПО

Выполнили:

Кузнецова А. А.

Куренный Г. И.

Левченко А. А.

Сысоев К. В.

Чен П. Я.

Б20-504

Деревянко А.К.

Б20-564

Проверяющий:

Козырев В. П.

Содержание

План разработки ПО	3
1. Стандарты	3
2. Модель жизненного цикла ПО	3
3. Разработка требований	4
3.1. Цели разработки требований	4
3.2. Входные данные процесса разработки требований к ПО	4
3.3. Выходные данные процесса разработки требований к ПО	5
3.4. Мероприятия процесса разработки требований к ПО	5
4. Проектирование	5
4.1. Цели проектирования	5
4.2. Входные данные процесса проектирования	6
4.3. Выходные данные процесса проектирования	6
4.4. Мероприятия процесса проектирования	6
5. Кодирование	7
5.1. Цели процесса кодирования	7
5.2. Входные данные процесса кодирования	7
5.3. Выходные данные процесса кодирования	7
5.4. Мероприятия процесса кодирования	7
6. Среда разработки	8
6.1. Разработка требований	8
6.2. Проектирование	8
6.3. Разработка	8

План разработки ПО

В данном разделе будет выбрана модель жизненного цикла, выбранная для разработки Системы, а также описано взаимодействие между различными процессами разработки.

1. Стандарты

При разработке ПО необходимо следовать следующим стандартам:

1. Стандарт на разработку требований к ПО
2. Стандарт на программирование

2. Модель жизненного цикла ПО

Для данного проекта была выбрана инкрементная модель жизненного цикла. Она подходит для разработки систем развертывания проектов и приложений, так как позволяет постепенно внедрять и тестировать функционал в реальной среде.

В рамках инкрементной модели разрабатываемое ПО делится на небольшие инкременты или фазы. Каждая фаза включает в себя анализ, проектирование, разработку и тестирование определенных функциональных возможностей или модулей системы.

Особенности инкрементной модели жизненного цикла ПО:

1. Итерации: Разработка ПО выполняется через серию повторяющихся итераций или циклов. Каждая итерация добавляет новые функции или улучшения к системе.
2. Постепенное увеличение функциональности: Каждая итерация добавляет новый функционал к системе, поэтому по мере продвижения разработки система постепенно становится все более полной и функциональной.
3. Обратная связь от пользователей: Возможность предоставления рабочих версий системы пользователю на ранних стадиях позволяет собирать обратную связь, учесть требования пользователя и вносить корректировки в последующие итерации.
4. Гибкость и адаптивность: Инкрементная модель позволяет гибко реагировать на изменения в требованиях пользователей или рынка и внедрять эти изменения в последующих итерациях.

5. Постепенное развитие системы: Разработка системы происходит путем постепенного расширения функциональности, что позволяет более эффективно управлять рисками и улучшить качество и стабильность системы.

Также в рамках инкрементной модели система может быстро внедряться и использоваться, инкрементная модель обеспечивает постепенное добавление функционала. Это позволяет начать использование системы развертывания на ранних этапах разработки, даже если функционал ограничен. Такой подход позволяет быстро получить обратную связь от пользователей и продолжать улучшать систему.

Инкрементная модель позволяет снижать риски, связанные с разработкой ПО. За счет разделения процесса на фазы итераций, регулярной проверки и совместной работы команды, возможность обнаружения и исправления ошибок раньше значительно увеличивается.

3. Разработка требований

Разработка требований к ПО является неотъемлемой частью жизненного цикла ПО, которая взаимодействует со всеми остальными процессами, определенными в рамках проекта. Процесс разработки требований использует входные данные, предоставляемые заказчиком (например, спецификации системных требований и архитектуры) и выходные данные, возникающие в ходе других процессов, связанных с текущим.

3.1. Цели разработки требований

Целью разработки требований к ПО является разработка требований высокого уровня.

3.2. Входные данные процесса разработки требований к ПО

Входные данные для процесса разработки требований к ПО включают в себя требования к системе, описание программно-аппаратных интерфейсов и архитектуры системы, План разработки ПО и Стандарты на разработку требований.

3.3. Выходные данные процесса разработки требований к ПО

Основным выходным результатом процесса разработки требований к ПО являются Требования к ПО, помимо этого, в результате данного процесса также могут появиться Сообщения о проблемах.

3.4. Мероприятия процесса разработки требований к ПО

Для завершения процесса разработки требований к ПО и достижения всех целей данного процесса необходимо выполнить следующее:

- Провести анализ требований, относящихся к функциональности и интерфейсу системы, с целью выявления неясностей и/или противоречий.
- Передать недостаточные или некорректные входные данные по обратной связи (на входы процессов-источников этих данных) для их уточнения или изменения.
- Внести каждое требование к системе, отнесенное к ПО, в требования высокого уровня.
- Определить требования высокого уровня, которые были связаны с введенными для предотвращения опасных ситуаций требований к системе.
- Удостовериться, что требования к ПО соответствуют Стандартам на разработку требований к ПО.
- Убедиться, что требования высокого уровня не включают в себя описания деталей реализации и верификации. Любые исключения должны быть обоснованы.
- Определить производные требования высокого уровня и обосновать причины их создания.
- Передать производные требования высокого уровня в процессы системы.

4. Проектирование

4.1. Цели проектирования

Процесс проектирования программного обеспечения имеет следующие цели:

1. Определение архитектуры программного обеспечения и перевод высокоуровневых требований в требования низкого уровня.
2. Определение дополнительных требований низкого уровня и их интеграция в процессы системы, включая оценку безопасности системы.

4.2. Входные данные процесса проектирования

Входными данными для процесса проектирования ПО являются Требования к ПО, План Разработки ПО.

4.3. Выходные данные процесса проектирования

Главным результатом этого процесса является Описание Архитектуры ПО, в котором дается описание архитектуры ПО и формулируются требования низкого уровня.

4.4. Мероприятия процесса проектирования

Процесс проектирования ПО считается завершенным, когда его цели достигнуты и достигнуты связанные с ним цели интегральных процессов. Мероприятия этого процесса включают в себя:

1. Требования низкого уровня и архитектура ПО, разработанные в процессе проектирования ПО, должны быть трассируемыми, верифицируемыми и непротиворечивыми.
2. Необходимо определить производные требования низкого уровня и причины их создания и проанализировать их, чтобы убедиться в том, что требования более высокого уровня не нарушены.
3. Мероприятия процесса проектирования ПО могут привести возможные виды отказов в ПО или, наоборот, устранить другие виды. Использование обособления или другого архитектурного решения в проекте ПО может изменить присвоенный уровень для некоторых компонентов ПО. В таких случаях необходимо определить дополнительные данные в виде производных требований и передать их в процессы системы, включая процесс оценки безопасности системы.
4. Должны быть определены интерфейсы между компонентами ПО в форме потока данных и потока управления таким образом, чтобы они были согласованными между компонентами.
5. Должен быть предусмотрен контроль потоков управления и данных, например, с помощью сторожевых таймеров, проверок на допустимость и перекрестного сравнения каналов, когда это диктуется требованиями в части обеспечения безопасности.

6. Реакции на отказные состояния должны удовлетворять требованиям в части обеспечения безопасности.
7. Недостаточные или некорректные входные данные, выявленные в процессе проектирования ПО, должны быть переданы в процессы ЖЦ системы, в процесс разработки требований к ПО или в процесс планирования ПО по обратной связи для уточнения или поправок.

5. Кодирование

В процессе кодирования ПО Исходный код реализуется на основе архитектуры ПО и требований низкого уровня.

5.1. Цели процесса кодирования

Цель процесса кодирования - разработать Исходный Код на основании требований низкого уровня.

5.2. Входные данные процесса кодирования

Входными данными процесса кодирования ПО являются требования низкого уровня и архитектура ПО, полученные из процесса проектирования ПО, а также План Разработки ПО и Стандарты на Программирование ПО. Исходный Код создается этим процессом на основе архитектуры ПО и требований низкого уровня.

5.3. Выходные данные процесса кодирования

Основным результатом этого процесса является Исходный Код

5.4. Мероприятия процесса кодирования

Процесс кодирования считается завершенным, когда его цели достигнуты и достигнуты связанные с ним цели интегральных процессов. Мероприятия этого процесса включают в себя:

1. Исходный Код должен реализовывать требования низкого уровня и соответствовать архитектуре ПО.
2. Исходный Код должен соответствовать Стандартам на Программирование ПО.
3. Недостаточные или некорректные входные данные, выявленные в процессе кодирования ПО, должны быть переданы в процесс разработки требований к

ПО, процесс проектирования ПО или в процесс планирования ПО по обратной связи для уточнения или поправок.

6. Среда разработки

6.1. Разработка требований

Для создания документации требований применяется Confluence, для форматирования и представления документации требований используется Microsoft Office Word.

6.2. Проектирование

Проектирование системы осуществляется с использованием CASE-инструментов в программе draw.io. Это приложение позволяет полно и наглядно представить архитектуру разрабатываемой системы, а также делиться ей с другими участниками проекта для обратной связи и согласования

6.3. Разработка

1. Для серверной части проекта используется JetBrains Goland.
2. Для клиентской части проекта используется VSCode.
3. Для хранения исходного кода используется система контроля версий GitLab.